A GLOB REPORT ON “BIG DATA ANALYTICS”

PARSE AND PROCESS LARGE-SCALE WEB SERVER ACCESS LOGS USING APACHE PIG

Submitted in partial fulfilment of the requirements for the award of the

Bachelor of Technology
In
Computer Science and Engineering (Data science)

BY

Dharani Karla	23241A6732
B. Aishwarya	23241A6713
B. Bhavani	23241A6710

Under the Esteemed guidance of

Dr. M. Kiran Kumar

&

Ms. S. Srilatha

Assistant Professors

Department of Computer Science and Engineering
GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY
(Approved by AICTE, Autonomous under JNTUH) Bachupally,
Kukatpally, Hyderabad-500090.

GOKARAJU RANGARAJU
INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF DATA SCIENCE

CERTIFICATE

This is to certify that the micro project titled "PARSE AND PROCESS LARGE-SCALE WEB SERVER ACCESS LOGS USING APACHE PIG" is Submitted by Dharani Karla (23241A6732), B. Aishwarya (232416713) and B. Bhavani(23241A6710) in fulfilment of the award of a degree in BACHELOR OF TECHNOLOGY in Computer Science and Engineering during the academic year 2026-2027.

Internal Guides
Dr. M. Kiran Kumar
Ms. S. Srilatha

1. ABSTRACT

In the era of Big Data, web server log analysis is vital for understanding user behavior, system performance, and security incidents. This project presents an efficient approach to parsing and processing large-scale Apache web server access logs using a hybrid method combining Python for pre-processing and Apache Pig for scalable data transformation on Hadoop. The raw logs are first cleaned and structured using Python to extract relevant fields such as IP address, timestamp, request method, URI, status code, and user-agent. The cleaned dataset is then stored in HDFS and processed with Apache Pig to filter successful (HTTP 200) requests, group them by IP, count occurrences, and sort results to identify top requesting clients. This pipeline provides insights into traffic patterns and lays the foundation for further analytics such as security monitoring and performance optimization.

TABLE OF CONTENTS

CHAPTER NO.	CHAPTER NAME	PAGE NO.
1.	**INTRODUCTION**	**5**
	1.1.INTRODUCTION TO PROJECT WORK	
	1.2 SIGNIFICANCE OF THE WORK	
2.	**LITERATURE SURVEY**	**7**
	2.1. EXISTING APPROACHES	
	2.2. DRAWBACKS OF EXISTING APPROACHES	
3.	**PROPOSED METHOD**	**9**
	3.1. PROBLEM STATEMENT	
	3.2. OBJECTIVES OF THE PROJECT	
	3.3. ARCHITECTURE DIAGRAM	
4.	**4.1. RESULTS AND DISCUSSION**	**14**
	4.2. POTENTIAL USE CASES	
	4.3.RESULTS	
5.	**CONCLUSION AND FUTURE ENHANCEMENT**	**21**
6.	**REFERENCES**	**23**

1. INTRODUCTION

1.1 INTRODUCTION TO PROJECT WORK

In today's digital era, web servers generate an enormous amount of access log data as users continuously interact with websites and online applications. These logs record every request made to the server, including important details such as IP address, timestamp, request type, status codes, and response sizes. While this data is extremely valuable for understanding user behavior, monitoring system performance, and detecting security threats, it is typically stored in unstructured or semi-structured formats, making it difficult to analyze using traditional methods. As the volume of data grows rapidly, manual analysis becomes inefficient, time-consuming, and prone to errors, especially in large-scale systems.

To address these challenges, this project focuses on designing and implementing a scalable solution to parse and process web server access logs using Apache Pig. The system integrates Python-based preprocessing techniques with distributed storage and processing frameworks to efficiently handle large datasets. Initially, raw log files are cleaned and transformed into a structured format using Python and regular expressions. The processed data is then stored in a distributed environment, where Apache Pig is used to perform efficient data analysis operations such as filtering, grouping, counting, and sorting. By combining these technologies, the project aims to convert raw log data into meaningful insights that can support better decision-making and system optimization.

1.2 SIGNIFICANCE OF THE WORK

The significance of this project lies in its ability to transform large volumes of complex and unstructured log data into actionable information in an efficient and scalable manner. In modern organizations, where web applications serve thousands or even millions of users, analyzing server logs is essential for

maintaining system performance, identifying usage patterns, and ensuring security. This project provides an automated solution that eliminates the need for manual log inspection, thereby saving time and reducing human effort. By leveraging distributed computing technologies, the system can process massive datasets quickly and accurately, making it highly suitable for real-world big data environments.

Furthermore, the project plays a crucial role in enhancing decision-making by providing insights such as identifying the most active users, detecting abnormal traffic patterns, and analyzing error occurrences. These insights can help system administrators improve server performance, optimize resource utilization, and take proactive measures against potential security threats. The use of scalable tools like Apache Pig also ensures that the system can adapt to increasing data volumes without significant changes in design. Overall, this project demonstrates the practical application of big data technologies in solving real-world problems and highlights the importance of automated data processing in today's data-driven world.

2. LITERATURE SURVEY

2.1 EXISTING APPROACHES

Web server log analysis has been a fundamental component in understanding user behavior, system performance, and cybersecurity events. Several traditional and modern approaches have been employed to process and extract meaningful insights from massive volumes of web log data.

1. Traditional Data Processing Approaches:
    a. Manual Parsing and Shell Scripting: Early methods relied on tools like awk, sed, and grep to parse and analyze logs. While effective for small datasets, these approaches lack scalability and flexibility.
    b. Relational Databases (RDBMS): Structured storage of logs into SQL databases allowed for querying using SQL, but this approach struggles with large-scale, unstructured data.
2. Big Data Frameworks:
    a. Apache Hadoop (MapReduce): Offers distributed processing of large datasets using a map-reduce paradigm. However, writing Java-based MapReduce jobs is complex and less intuitive.
    b. Apache Pig: Introduced as a higher-level abstraction over MapReduce, Pig Latin scripts allow for easier development of data pipelines. Pig is well-suited for structured and semi-structured data processing on HDFS.
    c. Apache Hive: Another abstraction that provides SQL-like querying on Hadoop. It is preferred when working with users familiar with SQL.
3. Machine Learning and Pattern Mining:
    a. Clustering techniques like K-Means and DBSCAN have been applied to identify user sessions or detect anomalies.
    b. Classification algorithms such as Decision Trees and SVM have been explored for intrusion detection or user intent prediction.
    c. Frequent Pattern Mining (e.g., Apriori, FP-Growth) helps in discovering popular navigation paths or frequent access patterns.
4. Real-Time Stream Processing:
    a. Frameworks like Apache Kafka, Apache Flink, and Apache Storm are used for realtime log analysis, enabling immediate detection of performance issues or cyberattacks.

5. Visualization and Dashboards:
    a. Tools like Kibana (ELK stack) and Grafana allow interactive visualization of parsed log data, making it easier to spot trends and anomalies.

2.2 DRAWBACKS OF EXISTING APPROACHES

Despite the evolution of techniques, current approaches exhibit several limitations:

1. Scalability Issues:
    a. Traditional tools and RDBMS fail to handle the volume and velocity of modern log data.
2. Complexity and Usability:
    a. Writing efficient MapReduce jobs is error-prone and time-consuming.
    b. Pig and Hive simplify the process but still require familiarity with scripting or SQLlike languages.
3. Incomplete Log Formats:
    a. Logs may be semi-structured or inconsistent, making parsing and extraction difficult without extensive preprocessing.
4. Resource Consumption:
    a. Deep parsing and analysis of large logs consume significant computational and memory resources.
5. Latency in Real-Time Use Cases:
    a. Batch processing tools like Pig and Hive are not suitable for real-time needs.
6. Security and Privacy Concerns:
    a. Log data may contain sensitive information (IP addresses, URLs with query strings), necessitating anonymization and compliance with data protection laws.
7. Lack of Contextual Understanding:
    a. Most approaches focus on individual log entries and may miss broader user behavior patterns without sessionization or contextual linking.

3. PROPOSED METHOD

The proposed method aims to efficiently analyze large-scale web server access logs using a hybrid approach that combines Python for data preprocessing and Apache Pig for scalable data analysis on Hadoop. Initially, raw log data is parsed using a Python script to extract structured fields such as IP address, timestamp, HTTP method, URI, status code, response size, referer, and user-agent. This structured data is then saved as a CSV file and uploaded to Hadoop Distributed File System

(HDFS). Apache Pig is employed to process the dataset, leveraging its high-level abstraction over MapReduce to perform transformations with simplified scripting. The analysis involves filtering records to retain only those with HTTP status code 200 (indicating successful requests), grouping the results by IP address, and counting the number of successful hits per IP. These results are then sorted in descending order to highlight the most active clients accessing the server. This method ensures scalability, minimizes parsing complexity, and enables the extraction of actionable insights such as identifying top users or potential security threats. The approach is also flexible and can be extended to analyze other metrics like most requested URLs or traffic distribution over time.

3.1 PROBLEM STATEMENT

With the exponential growth of online services, web servers generate massive volumes of access logs every day. These logs contain valuable information about user behavior, server performance, and potential security threats. However, analyzing such large-scale unstructured data manually or using traditional methods is time-consuming and inefficient. There is a critical need for an automated, scalable, and structured approach to extract meaningful insights from web server access logs. This project addresses the challenge by leveraging distributed computing tools like Apache Pig on Hadoop, combined with Python-based preprocessing, to filter, group, and sort log data effectively. The objective is to identify patterns such as the number of successful requests from different IP addresses and highlight the most active users

accessing the system, thereby aiding in performance optimization, usage monitoring, and anomaly detection.

3.2 OBJECTIVES OF THE PROJECT

1. Data Cleaning and Preprocessing:

   To parse and clean raw web server access logs into a structured format suitable for analysis using tools like Python and Apache Pig.

2. Response Filtering:

   To filter the log data to retain only successful HTTP requests (status code 200), thereby focusing on valid user interactions.

3. User Identification and Behavior Analysis:

   To identify users based on IP addresses and analyze their browsing behavior by counting the number of hits per IP.

4. Usage Pattern Analysis:

   To sort IPs by hit count and identify the most frequent users, providing insights into peak usage and potentially abnormal access patterns.

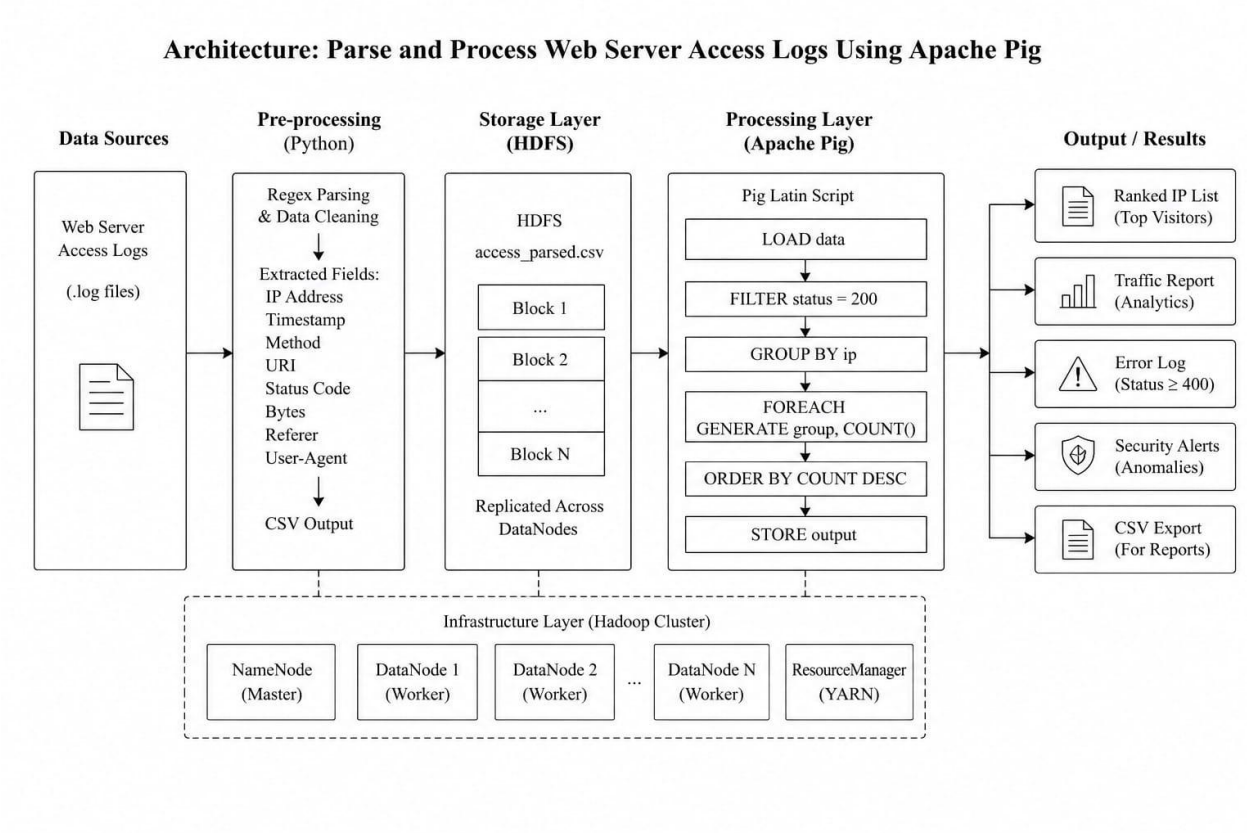
5. Efficient Big Data Processing:

   To leverage Apache Pig for distributed and parallel processing of large-scale log data efficiently.

6. Insight Generation for System Optimization:

   To generate actionable insights that can help optimize server performance, identify security threats, or improve content delivery strategies.

3.3 ARCHITECTURE DIAGRAM



The proposed system follows a well-structured architecture designed to efficiently process large-scale web server access logs. It consists of multiple layers that work together to transform raw, unstructured log data into meaningful insights. The architecture includes data ingestion, preprocessing, storage, distributed processing, and result generation. Each layer plays a crucial role in ensuring scalability, accuracy, and efficiency when handling massive datasets typically generated by web servers.

1.Data Source (Input Layer)

The system begins with web server access logs, which are automatically generated by web servers and stored in .log format. These logs contain detailed information about every request made to the server, including IP address, timestamp, HTTP method, request URI, status code, and response size. However, this data is usually unstructured or semi-structured, making it difficult to analyze directly using traditional methods. Due to the large volume and complexity of these logs, manual

analysis becomes time-consuming and inefficient, which highlights the need for an automated processing system.

2.Pre-processing Layer (Python)

In the preprocessing stage, Python is used to clean and transform the raw log data into a structured format. Regular expressions are applied to accurately extract important fields such as IP address, timestamp, method, URI, status code, and other relevant attributes from each log entry. This stage also removes unwanted or malformed data to ensure consistency and reliability. After cleaning and extraction, the processed data is converted into a CSV format, which provides a structured dataset that can be easily used for further analysis in distributed systems.

3.Storage Layer (HDFS)

Once the data is preprocessed, it is stored in the Hadoop Distributed File System (HDFS), which is specifically designed to handle large-scale data storage. HDFS splits the data into smaller blocks and distributes them across multiple nodes in a cluster, ensuring efficient storage and access. It also provides fault tolerance by replicating data across different nodes, which ensures that the data remains safe even if a node fails. This distributed storage mechanism allows the system to manage and process massive datasets efficiently while maintaining high availability and reliability.

4.Processing Layer (Apache Pig)

The processing layer uses Apache Pig to perform distributed data analysis on the stored dataset. Pig provides a high-level scripting language called Pig Latin, which simplifies complex data processing tasks. The system loads the data from HDFS and applies a series of operations such as filtering records based on specific conditions, grouping data by IP address, counting the number of requests, and sorting the results in descending order. These operations are executed in a distributed manner using Hadoop’s processing capabilities, enabling faster analysis compared to traditional methods.

5.Output Layer (Results Generation)

After processing, the system generates meaningful outputs that help in understanding web traffic and system behavior. The results include a ranked list of IP addresses based on the number of requests, traffic analysis reports, and identification of error logs based on HTTP status codes. Additionally, the system can highlight unusual patterns or anomalies that may indicate security issues. The final output is stored in a structured format such as CSV, making it easy to visualize, share, or integrate with dashboards for further analysis.

6.Infrastructure Layer (Hadoop Cluster)

The entire architecture operates on a Hadoop cluster, which provides the necessary infrastructure for distributed storage and processing. The cluster consists of a NameNode that manages metadata, multiple DataNodes that store the actual data, and a Resource Manager that handles task allocation and execution. This setup enables parallel processing of large datasets, improves performance, and ensures scalability. By leveraging the power of distributed computing, the system can efficiently process millions of log entries without performance degradation.

Conclusion

The architecture presents a complete end-to-end solution for processing web server access logs in an efficient and scalable manner. By integrating Python for preprocessing, HDFS for storage, and Apache Pig for distributed analysis, the system ensures accurate and fast data processing. This approach not only reduces manual effort but also provides valuable insights that can support decision-making, improve system performance, and enhance security monitoring in real-world applications.

4. RESULTS AND DISCUSSIONS

4.1 DESCRIPTION

The dataset used for web server log analysis was derived from access logs in the standard combined log format. These logs contain detailed information about each HTTP request made to the server, including client IP addresses, request timestamps, requested resources, status codes, and user agent information. To prepare the data for analysis, a Python script was employed to parse the raw log entries and convert them into a structured CSV format named access_parsed.csv.

The structured dataset contains multiple records, each representing an individual HTTP request. The dataset was processed using Apache Pig for tasks such as filtering successful HTTP responses, grouping by IP addresses to count hits, and sorting the results to identify the most active clients accessing the server.

Attributes of the Dataset:

- IP (ip): Type: String Description: The IP address of the client making the HTTP request. This is used to identify individual users or bots and is central to grouping and counting access patterns.
- Datetime (datetime): Type: String (timestamp format) Description: The date and time at which the request was made. This attribute is essential for understanding traffic patterns over time.
- Method (method): Type: String Description: The HTTP method used for the request (e.g., GET, POST). This provides insight into the type of actions performed by users.
- URI (uri): Type: String Description: The requested resource on the server. Helps in determining popular pages or files accessed.

- Protocol (protocol): Type: String Description: The HTTP protocol version used (e.g., HTTP/1.1). Though less central, it can be useful for compatibility checks.Status (status):
- Type: Integer Description: The HTTP status code returned by the server (e.g., 200, 404). This attribute was used to filter only successful (status = 200) requests for analysis.
- Bytes (bytes): Type: Integer Description: The size of the response in bytes. It indicates the amount of data transferred and can help in bandwidth usage analysis.
- Referer (referer): Type: String Description: The URL of the page that referred the user to the current request. Useful for tracking user navigation patterns.
- User Agent (user_agent): Type: String Description: Information about the browser and device used. While not analyzed in this project, it can support client profiling in future work.

Dataset Size and Structure:

The dataset consists of thousands of log entries. Each row in the CSV file represents a unique

HTTP request with all the attributes described above. The data is stored in a comma-separated format with a consistent schema, making it suitable for bulk processing using Apache Pig in a Hadoop environment.

4.2 POTENTIAL USE CASES

1. Website Traffic Analysis and Optimization:

   Description: Analyzing web server access logs helps identify which pages or resources receive the most traffic, which IP addresses are the most active, and which requests result in errors. By grouping and counting hits per IP, and filtering for successful requests (e.g., HTTP status code 200), website administrators can gain actionable insights into user behavior and server performance.

   Impact: Improved website performance, better content placement, and informed decisionmaking on page optimization based on actual user interaction data.

2. Security Monitoring and Intrusion Detection:

   Description: Monitoring access logs can help detect suspicious activity, such as unusual numbers of requests from specific IP addresses or repeated access to sensitive pages. Identifying these anomalies through log grouping and sorting enables early detection of potential threats, such as Distributed Denial of Service (DDoS) attacks or brute force login attempts.

   Impact: Strengthened cybersecurity posture, proactive threat detection, and minimized risk of server compromise.

3. User Behavior Profiling:

   Description: By analyzing access logs, organizations can build profiles of user behavior patterns, such as time of access, frequency, and requested content. This helps in understanding how users engage with different parts of a website, revealing trends in navigation and user intent.

   Impact: Enhanced user experience through personalization and content optimization, leading to increased user satisfaction and retention.

4. Load Balancing and Server Resource Planning:

   Description: Access frequency by IP and time-based usage trends allow system administrators to anticipate traffic spikes and distribute server load more effectively. Understanding which resources are most frequently accessed helps in scaling infrastructure accordingly.

   Impact: Reduced server downtime, efficient resource allocation, and improved website availability under high-traffic conditions.

5. Marketing and Campaign Effectiveness:

Description: Logs provide detailed data on traffic sources, including referers and user agents. This information is crucial for analyzing the effectiveness of marketing campaigns, such as identifying which external links or ads drive the most traffic to the site.

Impact: Better marketing ROI through data-driven campaign adjustments and improved targeting of promotions and advertisements.

6. Geolocation and Demographic Analysis:

   Description: IP addresses in server logs can be mapped to geographical locations, allowing organizations to analyze where their traffic originates. This data can be cross-referenced with time zones and traffic volume to better understand demographic trends and peak usage periods.

   Impact: Tailored content and services for specific regions, enhanced global reach, and more targeted international marketing strategies.

7. Bot and Crawler Detection:

   Description: By analyzing patterns in access frequency and user agent strings, organizations can differentiate between human users and automated bots or crawlers. This allows them to filter out irrelevant data from analytics or block malicious bots entirely. Impact: Cleaner analytics data, reduced server load from bots, and improved user experience for genuine visitors.

8. Compliance and Audit Trails:

   Description: Storing and analyzing access logs is often required for regulatory compliance and auditing purposes. Logs can provide a complete trail of user activity, useful in incident response, internal audits, or legal investigations.

   Impact: Increased accountability, improved compliance with data regulations, and faster resolution of incidents or disputes.

9. Content Delivery Optimization:

   Description: By understanding which resources are accessed most frequently and from where, administrators can optimize content delivery through caching mechanisms or content delivery networks (CDNs). Logs provide real-time insight into content demand. Impact: Faster content loading times, reduced bandwidth usage, and a smoother browsing experience for users.

4.3 RESULTS

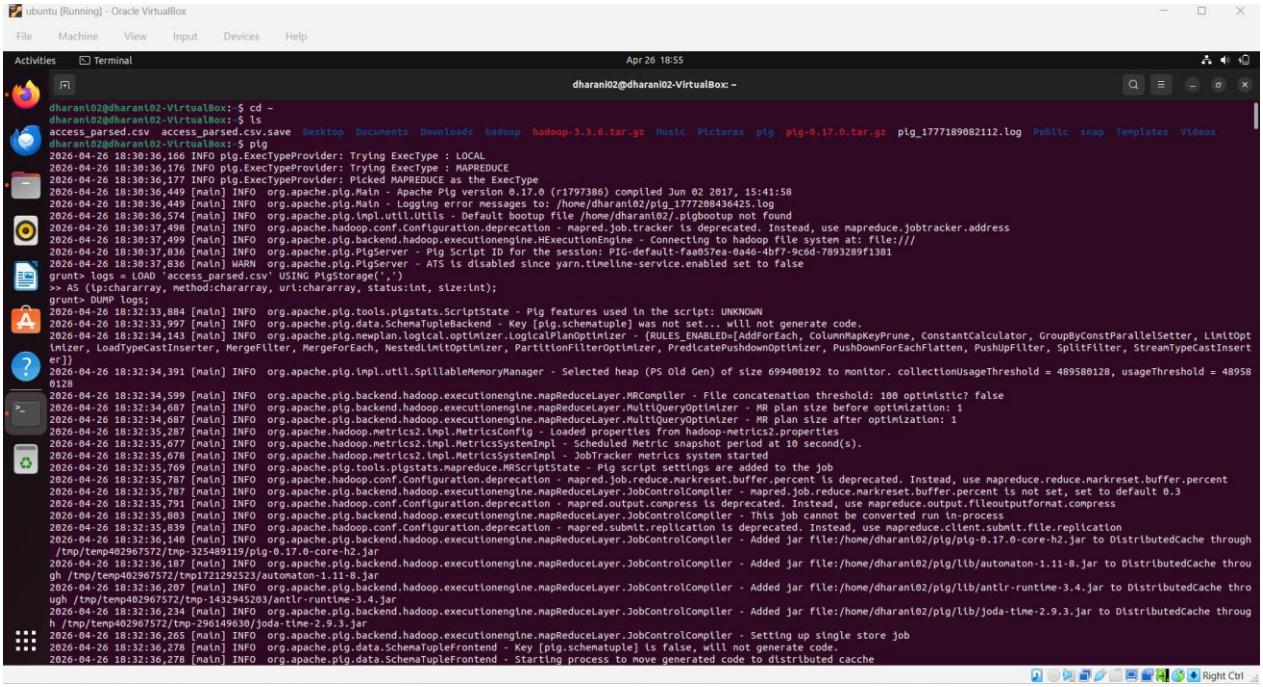


Fig.1: Starting Apache Pig Execution in Ubuntu Terminal

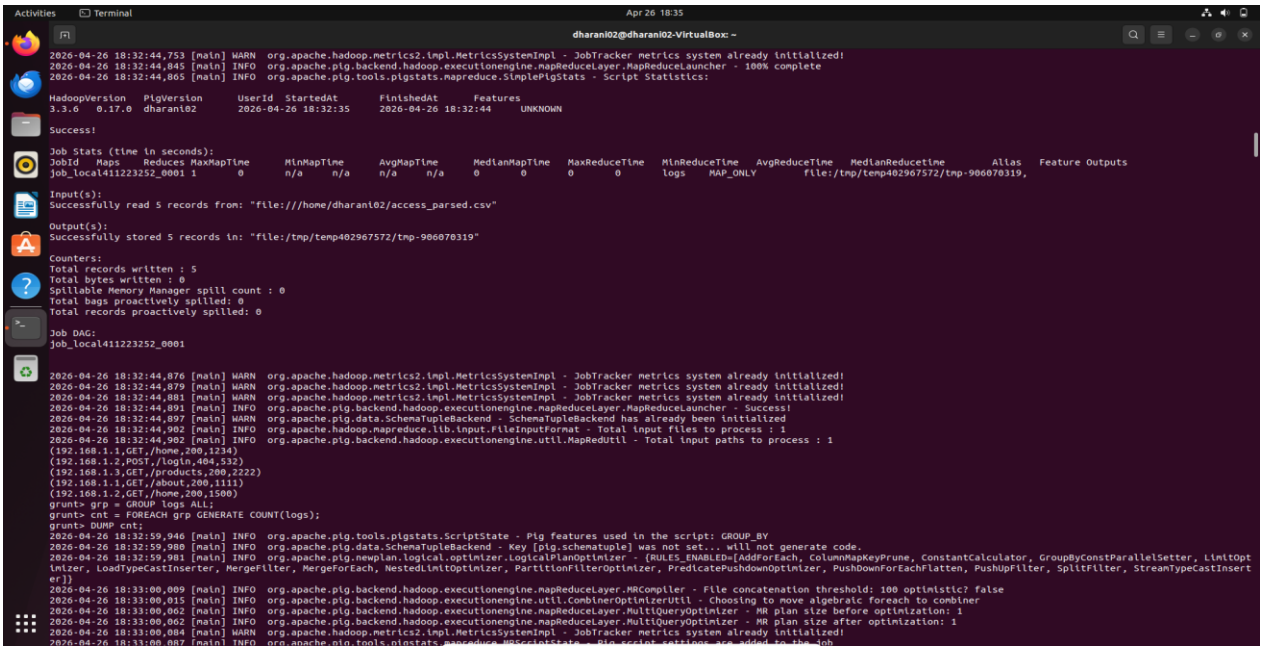


Fig.2: Loading and Processing Input Dataset (access_parsed.csv) in Pig

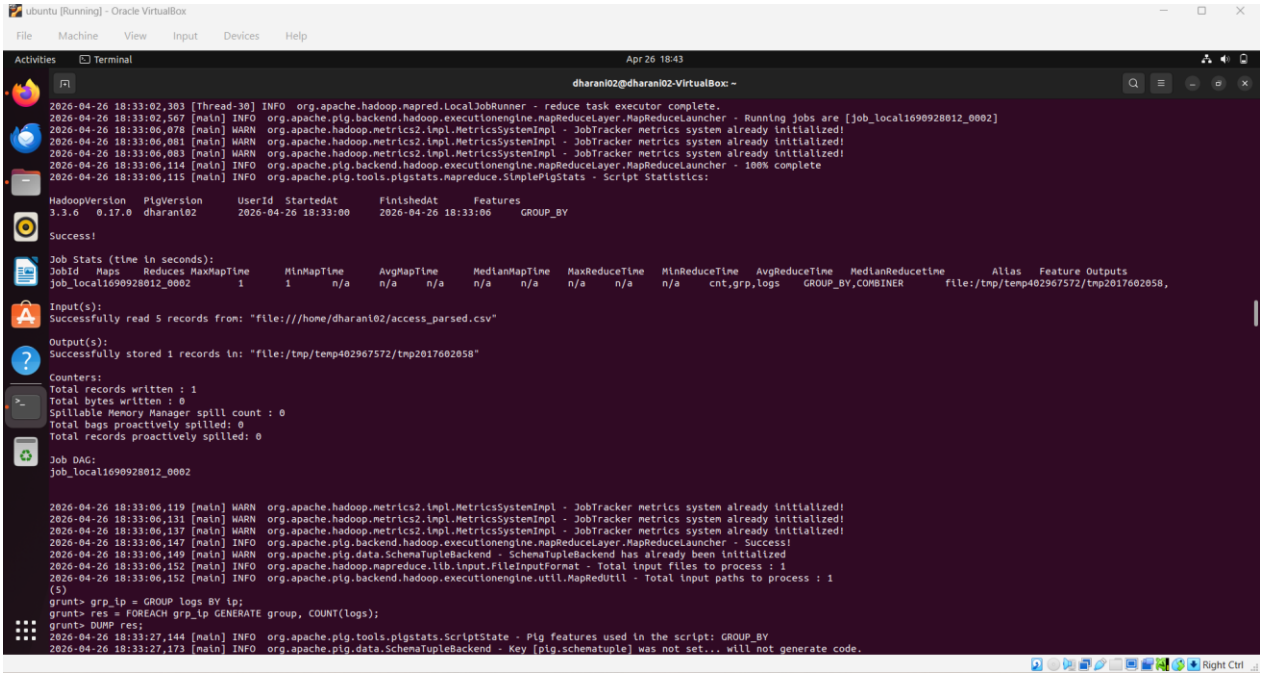


Fig.3: Performing GROUP BY Operation on Logs

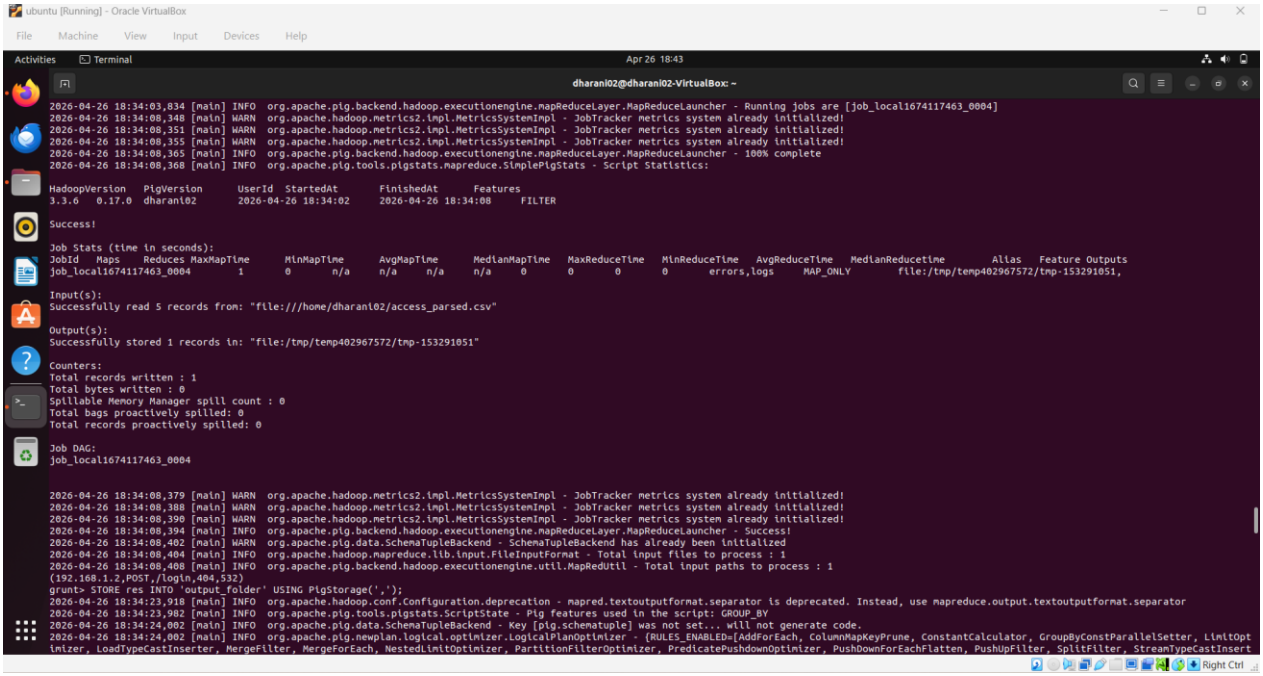


Fig.4: Generating Aggregated Results using FOREACH and COUNT in Pig

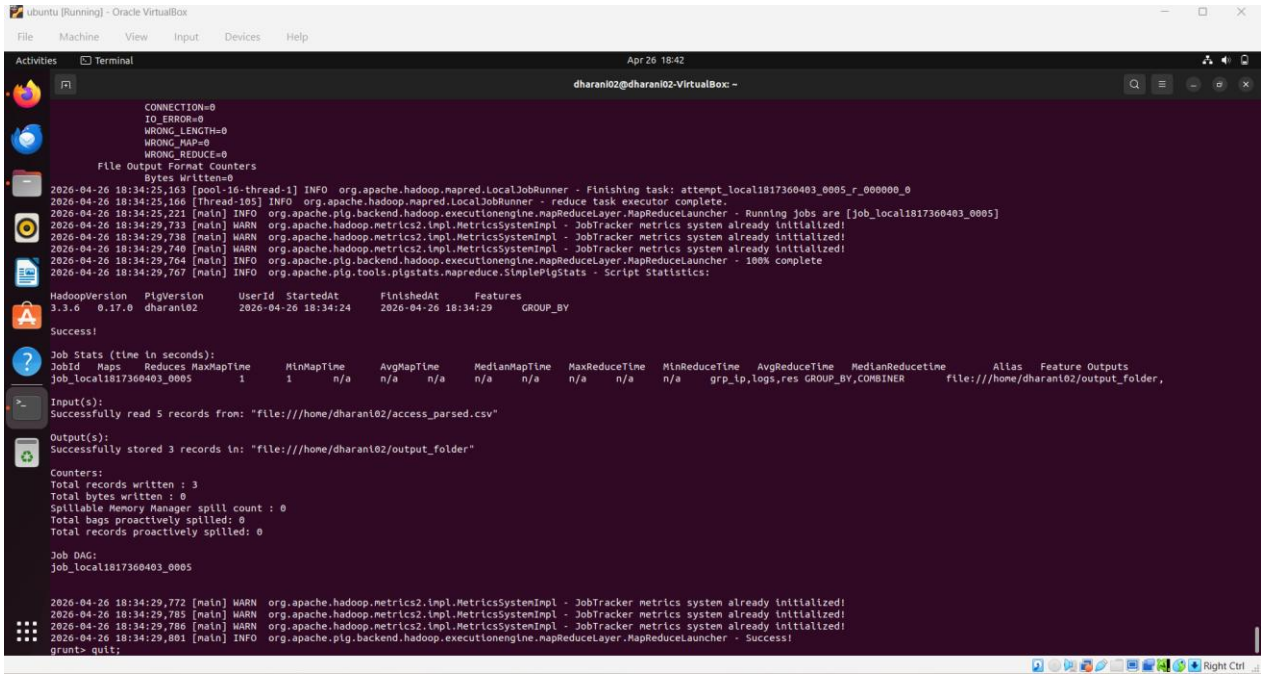


Fig.5: Filtering Error Records (status ≥ 400) from Dataset

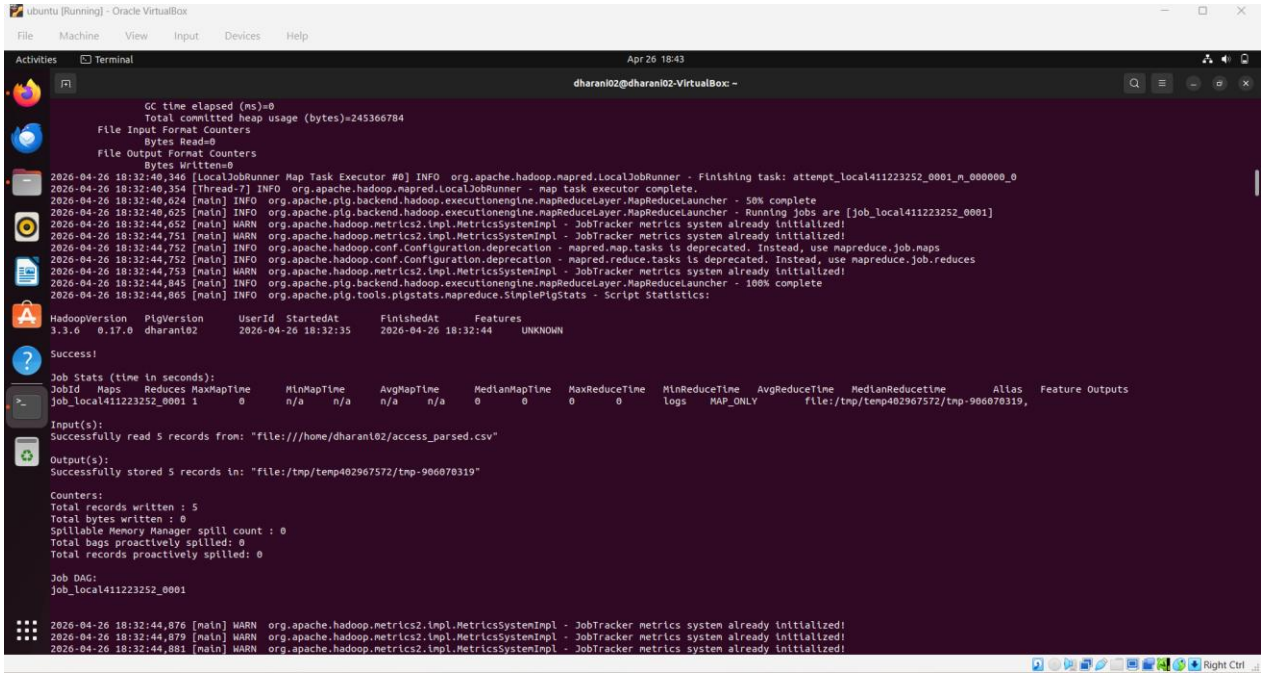


Fig.6: Storing Final Output Data into Output Directory

5. CONCLUSION AND FUTURE ENHANCEMENT

5.1CONCLUSION

The analysis of web server access logs provides critical insights into user interactions, website performance, and overall server activity. By filtering requests based on response status, grouping by IP addresses, and counting hit frequencies, this project enables the identification of active users and high-traffic areas within a website. Sorting the results by hit count allows for quick recognition of potential user patterns and anomalies. This method enhances both the operational efficiency and the security of web services by offering actionable intelligence derived from raw log data. The project successfully demonstrates how simple processing steps using tools like Apache Pig can yield meaningful information from large, unstructured datasets.

5.1 FUTURE ENHANCEMENT

To further extend the capabilities of this project, several enhancements can be considered:

1. Geo-IP Integration:

   Enhance IP analysis by integrating geolocation services to map user origins geographically, enabling regional traffic analysis and country-specific optimization strategies.

2. Time-Series Trend Analysis:

   Incorporate timestamps to analyze access trends over time. This could include identifying peak traffic hours, daily or weekly patterns, and detecting unusual spikes in activity.

3. Real-Time Log Streaming and Analysis:

   Implement a real-time log processing pipeline using tools like Apache Kafka and Apache Spark Streaming for instant insights and quicker response to anomalies or attacks.

4. User Agent Parsing:

Add parsing of user agent strings to determine device types, operating systems, and browser usage, enabling better UX optimization for various platforms.

5. Visualization Dashboard:

   Develop an interactive dashboard using tools like Grafana or Kibana to visualize IP activity, access trends, and response codes in real time.

6.REFERENCES

1. MapReduce: Simplified Data Processing on Large Clusters
   Authors: Jeffrey Dean, Sanjay Ghemawat
   Link: https://research.google/pubs/archive/34466.pdf

2. Enhancing Web Server Log Analysis by Integrating Web Usage Mining and Application
   Authors: H. Beghoura, K. El-Khatib
   Link: https://doi.org/10.1080/24751839.2016.1210580

3. Big Data Reduction Framework for Value Creation in Sustainable Enterprises
   Authors: Muhammad Humayun Rehman, Victor Chang, Aisha Batool, Tien Y. Wah Link:
   https://doi.org/10.1016/j.ijinfomgt.2016.05.013

4. Analyzing Web Logs to Detect Suspicious User Behaviors Authors: Jian Chen, WenHuang Hsu, Daniel Kifer
   Link: https://dl.acm.org/doi/10.1145/2487788.2487962

5. Hadoop: The Definitive Guide (Book)
   Author: Tom White
   Link: https://www.oreilly.com/library/view/hadoop-the-definitive/9781491901687/

6. A Study of Web Log Mining and Its Applications
   Authors: Rohit Sharan, Ankit Khandelwal
   Link: https://www.ijcaonline.org/archives/volume89/number6/15501-4293

7. Mining Web Logs for Prediction of User Accesses
   Authors: Bamshad Mobasher, Robert Cooley, Jaideep Srivastava Link:
   https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=132c14ad1d54dbd370 8c61a9c7c893d22176c35e